

GUÍA DE ESTUDIO

Linux y scripting en Bash

Análisis de mis scripts · Comandos · Bucles · Errores típicos

Implantación de Sistemas Operativos (ISO)
CFGS Administración de Sistemas Informáticos en Red (ASIR)

*Basada en mis propios scripts y trabajos de UD3, UD4, UD5 y UD6
Repaso para el examen*

Índice

0. Qué he analizado	3
0.1 Archivos de Linux revisados	3
0.2 Estructura típica de mis scripts	3
1. Análisis de mis comandos	4
1.1 Elementos del propio shell	4
1.2 Operadores dentro de []	5
1.3 Ficheros y directorios	5
1.4 Procesado de texto	6
1.5 Usuarios y grupos (UD5)	6
1.6 Paquetes (UD4)	8
1.7 Procesos y tareas programadas (UD6)	8
1.8 Colores ANSI que usé en los menús	9
2. Guía rápida de bucles en Bash	10
2.1 while condicional	10
2.2 while para leer un archivo línea a línea	11
2.3 Bucle for	11
2.4 Errores clásicos con bucles	12
2.5 Bucle dentro de mis scripts: mapa rápido	12
3. Fallos de mis scripts (por si piden depurar)	13
Otros detalles que también cuentan	14
4. Chuleta final y plan de repaso	16
4.1 Plantillas que debo saber escribir de memoria	16
4.2 Tabla resumen de parámetros especiales	17
4.3 Plan de repaso para esta noche (por prioridad)	17

0. Qué he analizado

Esta guía sale del análisis de todo el material de Linux del archivo **ISO.zip**: scripts de Bash, el CSV de usuarios y los PDFs de prácticas. Cada comando se explica con **mi propio código**, para que sea familiar el día del examen.

0.1 Archivos de Linux revisados

Unidad	Archivo	Qué hace
UD3	<code>script2.sh</code>	Número negativo, cero o positivo (<code>if/elif</code>).
UD3	<code>script3.sh</code>	Par o impar (módulo %).
UD3	<code>script4.sh</code>	Aprobado o suspenso según la nota.
UD3	<code>script5.sh</code>	Día laboral o fin de semana (<code>&&</code> , <code> </code>).
UD3	<code>script6.sh</code>	Mayor de tres números.
UD3	<code>script7.sh</code>	Mes → 1º o 2º semestre.
UD3	<code>script8.sh</code>	Año bisiesto.
UD3	<code>script9.sh</code>	Menú con <code>case</code> : crear directorio, crear archivo, multiplicar, salir. Usa <code>\$?</code> .
UD3	<code>script-DRV.sh</code>	Menú: listar directorio, multiplicar, <code>chmod</code> , <code>chown/chgrp</code> , crear archivo.
UD3	<code>permisos.sh</code>	Menú de permisos: mostrar, octal, <code>ugo+-</code> , <code>ugo=</code> , cambiar grupo.
UD3	<code>parametros2.sh</code>	Cambia propietarios de un archivo con confirmación S/N.
UD4	<code>gestionappslinux.sh</code>	Menú de gestión de software: <code>apt</code> , <code>dpkg</code> , <code>synaptic</code> , reconfigurar.
UD5	<code>script-usuarios-DRV.sh</code>	Crea usuarios en masa leyendo <code>usuarios.csv</code> .
UD5	<code>script-eliminar-usuarios-DRV.sh</code>	Menú para eliminar todos los usuarios del CSV o solo uno.
UD5	<code>script-investigacion.sh</code>	Registro de actividad: <code>last</code> , <code>who</code> , <code>w</code> , <code>id</code> .
UD5	<code>usuarios.csv</code>	30 usuarios: nombre, apellido, DNI, teléfono, correo, usuario.
UD5	PDFs de usuarios y grupos I y II	Ejercicios con <code>groupadd</code> , <code>useradd</code> , <code>usermod</code> , <code>userdel</code> , etc.
UD4	PDF de gestión de software	<code>dpkg</code> , <code>apt</code> , <code>apt-get</code> , <code>aptitude</code> , claves, <code>sass</code> y <code>synaptic</code> .
UD6	PDF Ejercicios-Repaso	<code>ps</code> , <code>pstree</code> , <code>kill</code> , <code>nice</code> , <code>jobs</code> , <code>nohup</code> ...
UD6	PDF Timer Systemd	Unidades <code>.timer</code> y <code>.service</code> , comparativa con <code>cron</code> .

Sobre «credenciales y alertas»: en mis archivos no hay nada llamado así literalmente. Lo más parecido a *credenciales* es `chpasswd` (poner contraseñas) y `useradd -e/-f` (caducidad de cuentas). Lo más parecido a *alertas* son los timers de systemd (tareas programadas) y los mensajes de error con `$?`. Los PDFs de Windows Server, UD1 y UD2 no se incluyen porque no son de Linux.

0.2 Estructura típica de mis scripts

Casi todos mis scripts siguen el mismo esqueleto. Si lo entiendes, entiendes cualquier script mío:

```
#!/bin/bash                                # 1. Shebang: intérprete
clear                                       # 2. Limpiar pantalla
function mostrarMenu(){ ... }             # 3. Funciones (menú, salir, tareas)
if [ $# -eq 1 ]; then                     # 4. Validaciones: nº de parámetros
    if [ -f $1 ]; then                   #   el archivo existe
        if [ $UID -eq 0 ]; then         #   se ejecuta como root
            opcion=1
            while [ $opcion -ne 0 ] # 5. Bucle principal del menú
            do
                mostrarMenu
                read -p 'Opción: ' opcion
                case $opcion in          # 6. Selección de la acción
                    0) salir ;;
                    1) funcion1 ;;
                    *) echo 'Opción incorrecta' ;;
                esac
            done
        fi
    fi
fi
```

1. Análisis de mis comandos

Esta parte está dividida por categorías. En cada tabla verás **la función del comando y el uso que le di en mis scripts**.

1.1 Elementos del propio shell

Esto es lo que más preguntan en un examen de scripting: no son programas externos, son la sintaxis del lenguaje.

Elemento	Función	Mi uso real
<code>read -p 'texto' var</code>	Pide un dato y lo guarda en <code>var</code> . <code>-p</code> muestra un mensaje. <code>-r</code> evita que \ se interprete.	<code>read -p 'Selecciona una opción [1-6]: ' opcion</code> (script-DRV, permisos)
<code>echo</code>	Imprime texto. <code>-n</code> sin salto de línea. <code>-e</code> interpreta \t, \e[92m (colores).	<code>echo -n '.'</code> en la animación «SALIENDO...». <code>echo -e "\${rojo}ERROR\${normal}"</code>
<code>clear</code>	Limpiar la pantalla.	Primera línea de casi todos mis menús.
<code>sleep n</code>	Pausa <code>n</code> segundos (admite decimales).	<code>sleep 0.2</code> entre cada usuario creado.
<code>exit / return</code>	<code>exit</code> termina todo el script. <code>return</code> sale solo de la función.	<code>exit</code> en <code>salir()</code> . <code>return</code> en <code>borrarUNO</code> al responder «N».
<code>if / elif / else / fi</code>	Condicionales.	<code>script2.sh</code> : negativo, cero o positivo.
<code>case ... esac</code>	Menú de opciones. <code>;;</code> cierra cada caso. <code>*)</code> es «cualquier otra cosa».	<code>case \$opcion in 0) salir ;; 1) informacion \$1 ;; *) echo 'Debes ELEGIR...' ;; esac</code>
<code>[]</code> (test)	Evalúa condiciones (ver 1.2).	<code>if [-d \$directorio]</code>
<code>\$(())</code>	Aritmética entera.	<code>\$((\$numero % 2))</code> , <code>\$((\$valor1 * \$valor2))</code> , <code>contador=\$((\$contador + 1))</code>
<code>\$()</code>	Sustitución de comando: guarda su salida.	<code>lineas=\$(wc -l \$1 cut -d' ' -f1)</code>
<code>\$?</code>	Código de salida del último comando (0 = éxito).	<code>mkdir \$nombre1 2>/dev/null</code> y luego <code>if [\$? -eq 0]</code>
<code>\$#</code>	Número de parámetros recibidos.	<code>if [\$# -eq 1]</code> (el CSV)
<code>\$0, \$1, \$*</code>	Nombre del script, primer parámetro, todos los parámetros.	<code>echo 'Debes introducir: sudo '\$0' \$1</code>
<code>\$UID</code>	UID del usuario actual (0 = root).	<code>if [\$UID -eq 0]</code> para exigir <code>sudo</code> .
<code>function nombre(){ }</code>	Define una función. Se llama escribiendo su nombre.	<code>mostrarMenu()</code> , <code>salir()</code> , <code>informacion()</code> , <code>borrarTODOS()</code> , <code>borrarUNO()</code>
<code>2>/dev/null</code>	Descarta los mensajes de error (descriptor 2 = stderr).	<code>sudo useradd ... 2>/dev/null</code> ; el resultado lo compruebo con <code>\$?</code> .

Elemento	Función	Mi uso real
&& /	A && B: ejecuta B solo si A tuvo éxito. A B: B solo si A falló.	sudo chown \$own \$archivo && sudo chgrp \$grp \$archivo
(pipe)	La salida de un comando es la entrada del siguiente.	cut -d"," -f1,2,5,6 \$1 column -t -s ","
> / >>	Redirige salida a archivo. > sobrescribe, >> añade.	echo \$info >> \$archivo
<	El archivo es la entrada estándar del comando o bucle.	done < \$1 al final de mis while read.
\${var}	Expansión de variable delimitada.	\${verde}'✓'\${normal} para los colores.

1.2 Operadores dentro de []

Tipo	Operador	Significado	Ejemplo mío
Números	-eq -ne	igual, distinto	[\$lineas -ne 0]
Números	-lt -le	menor, menor o igual	[\$numero -lt 0]
Números	-gt -ge	mayor, mayor o igual	[\$nota -ge 5]
Texto	== !=	igual, distinto	[\$si_no == 'S']
Texto	-z / -n	cadena vacía / no vacía	[-z \$?]
Fichero	-f	existe y es un fichero	[-f \$1]
Fichero	-d	existe y es un directorio	[-d \$directorio]
Fichero	-e	existe (fichero o directorio)	[-e \$nombre2.\$extension]
Lógicos	&& -a -o	Y, O (fuera de corchetes o dentro)	[\$dia -ge 8] [\$dia -le 0] y [\$nota -lt 0 -o \$nota -gt 10]

Regla de oro: números → -eq -lt -gt; texto → == !=. Espacios obligatorios dentro de los corchetes: [\$a -eq 1] funciona; [\$a -eq 1] falla.

1.3 Ficheros y directorios

Comando	Función	Mi uso real
ls	Lista contenido. -l detallado, -d el directorio en sí, -R recursivo, -l uno por línea.	ls -l \$archivo tras chmod. ls -ld \$directorio. ls -R en permisos.sh. ls -l *.deb.
cat	Muestra el contenido de un fichero.	cat \$archivo tras escribirlo. cat /etc/passwd grep \$usuario. cat grupo.
touch	Crea un fichero vacío (o actualiza su fecha).	touch \$nombre2.\$extension
mkdir	Crea un directorio.	mkdir \$nombre1 2>/dev/null y compruebo \$?.
cd	Cambia de directorio.	cd \$ruta 2>/dev/null en el script de dpkg.
chmod	Cambia permisos: octal (755) o simbólico (u+x, g=rw).	chmod \$octal \$archivo; chmod \$permiso \$archivo; chmod \$personas=''\$permisos \$archivo.
chown	Cambia el propietario (usuario:grupo).	sudo chown \$own':'\$own \$arch (parametros2.sh).
chgrp	Cambia el grupo propietario.	sudo chgrp \$grupo \$archivo (permisos.sh, script-DRV).
whereis	Localiza binario, código fuente y manual de un programa.	whereis \$app_ins tras instalar o desinstalar.

Repaso rápido de permisos

Valor	4	2	1
Permiso	r (leer)	w (escribir)	x (ejecutar)

Tres dígitos en octal: **usuario, grupo, otros**. Ej.: `chmod 754 f` → usuario `rw` (7), grupo `r-x` (5), otros `r--` (4). En simbólico: `u` usuario, `g` grupo, `o` otros, `a` todos; `+` añade, `-` quita, `=` fija exactamente.

1.4 Procesado de texto

Comando	Función	Mi uso real
<code>cut</code>	Extrae columnas. <code>-d</code> delimitador, <code>-f</code> campos.	<code>cut -d"," -f1,2,5,6 \$1</code> → nombre, apellido, correo y usuario del CSV.
<code>column -t -s ", "</code>	Alinea la entrada en forma de tabla.	Para mostrar el listado de usuarios bien alineado.
<code>wc -l</code>	Cuenta líneas.	<code>wc -l \$1</code> para saber cuántos usuarios hay en el CSV.
<code>grep</code>	Filtra líneas con un patrón. <code>-q</code> silencioso, <code>-w</code> palabra completa, <code>-v</code> invierte.	<code>apt show \$app grep -q 'Package:'</code> ; <code>apt list --installed grep -w "\$app"</code> ; <code>dpkg -l grep \$prg</code> .
<code>sed</code>	Editor de flujo.	<code>sed -n '1p;2p;4p;5p'</code> (solo esas líneas) y <code>sed 's/[_0-9].*//'</code> (borra desde el primer <code>_</code> o dígito para sacar el nombre del paquete).
<code>sort</code>	Ordena. <code>-r</code> inverso, <code>-k3</code> por columna 3.	<code>ps -ux sort -r -k3 > mis_procesos.txt</code>
<code>tail</code>	Muestra las últimas líneas.	<code>tail -3 /etc/passwd</code> y <code>tail -3 /etc/group</code> (los 3 últimos usuarios y grupos).
<code>wc -w</code>	Cuenta palabras.	<code>pidof -c bash wc -w</code> (cuántos bash hay activos).

1.5 Usuarios y grupos (UD5)

Comando	Función	Mi uso real
<code>sudo</code>	Ejecuta con privilegios de root.	Delante de casi toda la administración.
<code>useradd</code>	Crea un usuario.	Ver desglose abajo.
<code>userdel -r</code>	Borra el usuario. <code>-r</code> borra también su home.	<code>sudo userdel -r \$usuario</code>
<code>groupadd</code>	Crea un grupo.	<code>sudo groupadd alumnos_sn 2>/dev/null</code>
<code>chpasswd</code>	Asigna contraseñas leyendo <code>usuario:clave</code> por stdin.	<code>echo '\$usuario:...' sudo chpasswd</code> (con error, ver cap. 3).
<code>id usuario</code>	Muestra UID, GID y grupos.	<code>id \$usuario</code> en script-investigacion.sh.
<code>who</code>	Usuarios conectados ahora.	Opción 2 de mi menú de investigación.
<code>w</code>	Como <code>who</code> pero con más detalle (qué hace cada usuario).	Si respondía «S» a «¿más información?».
<code>last [usuario]</code>	Historial de inicios de sesión.	<code>last \$usuario</code> y <code>last</code> a secas.
<code>whoami</code>	Muestra tu nombre de usuario.	<code>usuario=\$(whoami)</code>

Mi `useradd` desglosado

```
sudo useradd $usuario \
  -c "Usuario temporal Academia Sierra Nevada - [$nombre $apellido]" \  # comentario / nombre completo
  -d /home/aula3/$usuario \      # directorio home
  -m \                          # crear el directorio home
  -s /bin/bash \                 # shell de inicio
  -g alumnos_sn \                # grupo principal
  -e 2026-01-29 \                # fecha de caducidad de la CUENTA
  -f 0                           # días de gracia tras caducar la contraseña (0 = deshabilitar ya)
```

Otras opciones muy preguntadas

Opción	Significado
<code>-G g1,g2</code>	Grupos suplementarios al crear (<code>useradd -G</code>).
<code>-u 1500</code>	Fija el UID.
<code>-M</code>	No crear home.
<code>-s /usr/sbin/nologin</code>	Shell sin inicio de sesión interactivo.
<code>-e</code> vs <code>-f</code>	<code>-e</code> = fecha en la que caduca la cuenta. <code>-f</code> = días de gracia con contraseña caducada antes de bloquear la cuenta.

Comandos que piden los enunciados de UD5

En mis PDFs de usuarios y grupos I y II las respuestas eran capturas, pero estos son los comandos que resuelven cada paso. Conviene saberlos de memoria.

Tarea	Comando
Crear grupos	<code>sudo groupadd asir / sudo groupadd -g 2000 profesores</code>
Cambiar GID	<code>sudo groupmod -g 2001 asir</code>
Comprobar un grupo	<code>getent group asir</code> o <code>grep asir /etc/group</code>
Crear usuario con grupo principal	<code>sudo useradd -g goya juan</code> (sin <code>-m</code> no crea el home)
Con home y grupos extra	<code>sudo useradd -m -d /home/marta -g profesores -G goya,asir marta</code>
Poner nombre completo	<code>sudo usermod -c "Juan Pérez Pérez" juan</code>
Ver grupos de un usuario	<code>groups marta</code> o <code>id marta</code>
¿Tiene contraseña?	<code>sudo passwd -S juan</code> (P = tiene, L = bloqueada, NP = sin contraseña)
Poner contraseña	<code>sudo passwd marta</code> o <code>echo 'marta:marta22' sudo chpasswd</code>
Cambiar home (moviendo contenido)	<code>sudo usermod -d /home/profes -m marta</code>
Cambiar shell	<code>sudo usermod -s /bin/sh juan</code>
Entrar como otro usuario	<code>su - marta</code> (exit para volver)
Añadir a grupo suplementario	<code>sudo usermod -aG grupo1 usuario7</code> (-a = añadir, sin él se sustituyen)
Cambiar grupo principal	<code>sudo usermod -g grupo1 usuario1</code>
Cambiar UID	<code>sudo usermod -u 1234 usuario3</code>
Renombrar usuario	<code>sudo usermod -l domalonso usuario4</code>
Bloquear / desbloquear	<code>sudo usermod -L usuario2 / -U</code> ; comprobar con <code>passwd -S</code>
Caducidad de cuenta	<code>sudo useradd -e 2017-12-24 usuario7</code> o <code>sudo chage -E 2017-12-24 usuario7</code>
Días de gracia	<code>sudo useradd -e 2017-11-23 -f 5 usuario8</code>
Ver caducidad	<code>sudo chage -l usuario7</code>
Borrar grupo	<code>sudo groupdel temporal</code> (no deja si es grupo principal de alguien)
Borrar usuario con home	<code>sudo userdel -r juan</code>
Ver los últimos creados	<code>tail -3 /etc/passwd</code> y <code>tail -3 /etc/group</code>

Archivos de configuración que debo conocer

Archivo	Contiene
<code>/etc/passwd</code>	Cuentas: <code>nombre:x:UID:GID:comentario:home:shell</code>
<code>/etc/shadow</code>	Contraseñas cifradas y caducidades (solo root).
<code>/etc/group</code>	Grupos y sus miembros.
<code>/etc/adduser.conf</code>	Valores por defecto de <code>adduser</code> : <code>DHOME</code> , <code>DSHELL</code> , <code>FIRST_UID</code> , <code>LAST_UID</code> , <code>FIRST_GID</code> ...

Archivo	Contiene
/etc/login.defs	Políticas de contraseñas y cuentas: PASS_WARN_AGE, PASS_MAX_DAYS, UID_MIN...

Pregunta típica: «¿Qué pasa con los usuarios de un grupo secundario si lo borro?» Simplemente dejan de pertenecer a él. En cambio, un grupo que es **principal** de algún usuario **no se puede borrar** hasta eliminar a ese usuario o cambiarle el grupo.

1.6 Paquetes (UD4)

Comando	Función	Mi uso real
apt update	Refresca la lista de paquetes de los repositorios.	Opción 1 de mi menú apt.
apt show paquete	Muestra los detalles de un paquete (sirve para saber si existe).	sudo apt show \$app_ins 2>/dev/null grep -q 'Package:'
apt install	Instala un paquete y sus dependencias.	sudo apt install \$app_ins
apt list --installed	Lista los paquetes instalados.	... grep -w "\$app_unins"
apt purge	Desinstala y borra la configuración .	sudo apt purge \$app_unins
apt autoremove	Elimina dependencias huérfanas.	Lo usé con nombre de paquete, pero no lo necesita.
dpkg -i	Instala un archivo .deb local.	sudo dpkg -i \$arch_deb
dpkg -l	Lista paquetes instalados.	sudo dpkg -l grep \$prg_deb
dpkg -r	Desinstala conservando la configuración.	sudo dpkg -r \$unins_dpkg_1
dpkg --configure	Reconfigura un paquete ya instalado.	Opción 4 de mi menú.
synaptic	Gestor gráfico de paquetes (frontend de apt).	sudo synaptic

¿Qué quiero?	dpkg	apt
Instalar	dpkg -i archivo.deb (no resuelve dependencias)	apt install paquete (descarga y resuelve dependencias)
Quitar conservando config	dpkg -r	apt remove
Quitar y borrar config	dpkg -P	apt purge
Listar	dpkg -l	apt list --installed
Fuentes de software	—	/etc/apt/sources.list y /etc/apt/sources.list.d/

1.7 Procesos y tareas programadas (UD6)

Comando	Función y ejemplo de mis ejercicios
ps -aux	Todos los procesos del sistema, con detalle y sin terminal.
ps -U root	Procesos de un usuario.
ps -ux sort -r -k3 > mis_procesos.txt	Mis procesos ordenados por CPU, guardados en un fichero.
pstree -up \$\$	Árbol de procesos con usuarios y PID. \$\$ es el PID del shell actual.
pidof -c bash wc -w	PIDs de los bash activos y cuántos hay.
sleep 300 &	Lanza en segundo plano. \$! guarda el PID del último proceso en segundo plano.
kill \$!	Termina un proceso (SIGTERM). kill -STOP lo pausa, kill -CONT lo reanuda, kill -9 lo mata a la fuerza.

Comando	Función y ejemplo de mis ejercicios
<code>killall yes</code>	Envía la señal a todos los procesos con ese nombre. <code>killall -STOP yes</code> los pausa.
<code>nice -n 10 comando</code>	Lanza con una prioridad. Rango -20 (más prioridad) a 19 (menos).
<code>sudo renice -n -5 pid</code>	Cambia la prioridad de un proceso ya en marcha (bajar el valor requiere root).
<code>jobs, fg 2, bg 2</code>	Lista trabajos, trae el 2 a primer plano, lo reanuda en segundo plano. <code>Ctrl+Z</code> lo detiene.
<code>nohup comando &</code>	El proceso sobrevive al cierre de la terminal. La salida va a <code>nohup.out</code> .

Timers de systemd (alternativa a cron)

Un **timer** es el «cuándo» y el **service** el «qué». Van siempre en pareja en `/etc/systemd/system/`.

```
# /etc/systemd/system/git-pull.service
[Unit]
Description=Actualizar repositorio
After=network-online.target
Wants=network-online.target
[Service]
Type=oneshot
User=tu-usuario
ExecStart=/usr/bin/bash /ruta/absoluta/gitpull.sh

# /etc/systemd/system/git-pull.timer
[Unit]
Description=Ejecutar git-pull cada minuto
[Timer]
OnBootSec=1min          # 1 minuto después de arrancar
OnUnitActiveSec=1min    # y luego cada minuto
[Install]
WantedBy=timers.target

# Activar y comprobar
sudo systemctl daemon-reload
sudo systemctl enable --now git-pull.timer
systemctl list-timers
systemctl status git-pull.timer
journalctl -u git-pull.service
```

Directiva	Significado
<code>OnCalendar</code>	Fecha/hora exacta (reloj de pared): <code>DíaSemana Año-Mes-Día Hora:Min:Seg.</code>
<code>OnBootSec</code>	Tiempo transcurrido desde el arranque (temporizador monótono).
<code>OnUnitActiveSec</code>	Tiempo desde la última activación del servicio (repetición relativa).
<code>Persistent=true</code>	Si el equipo estaba apagado a la hora prevista, ejecuta al encender (equivalente a anacron).
<code>RandomizedDelaySec</code>	Retardo aleatorio para no saturar el sistema al arrancar.

Aspecto	cron	systemd timers
Sintaxis	Compacta, menos legible	Más verbosa, más legible
Dependencias	Ciego ante fallos de red o servicios	Espera a red y otros servicios (<code>After=</code> , <code>Wants=</code>)
Tareas perdidas	Requiere anacron	Integrado con <code>Persistent</code>
Depuración	Buscar en <code>/var/log/syslog</code>	<code>systemctl status</code> y <code>journalctl -u</code>

1.8 Colores ANSI que usé en los menús

```
verde='\e[92m'; rojo='\e[91m'; azul='\e[94m'; amarillo='\e[93m'; normal='\033[0m'
echo -e "Usuario ${amarillo}$usuario${normal} ${verde}CREADO ✓${normal}"
```

Siempre con `echo -e` y cerrando con `${normal}` para volver al color por defecto.

2. Guía rápida de bucles en Bash

Hay tres situaciones que debes distinguir. Cuando lo tienes claro, elegir el bucle es automático.

Situación	Bucle
Repetir hasta que el usuario elija salir o dé un dato válido (no sé cuántas vueltas)	<code>while [condición]</code>
Procesar un fichero línea a línea o por columnas	<code>while IFS=sep read -r ... done < fichero</code>
Recorrer una lista, un rango o ficheros (sé qué elementos hay)	<code>for ... in ...</code>

2.1 `while` condicional

Repite **mientras la condición sea verdadera**. La condición se evalúa **antes** de cada vuelta, así que si ya es falsa al principio, el bucle no se ejecuta ni una vez.

```
while [ condición ]
do
    comandos
done
```

Mi caso real: menú que se repite hasta elegir 0

```
opcion=1 # valor inicial que ENTRA en el bucle
while [ $opcion -ne 0 ]
do
    mostrarMenu
    read -p 'Selecciona una opción: ' opcion # aquí se modifica la variable
    case $opcion in
        0) salir ;;
        1) apt ;;
        2) dpkg ;;
        *) echo 'Opción incorrecta' ;;
    esac
done
```

Lógica clave: (1) inicializar la variable a un valor que haga entrar; (2) modificarla dentro del bucle; (3) que algún día la condición se vuelva falsa. Si falta el punto 2, el bucle es infinito (**Ctrl+C** para salir).

Otros patrones útiles

```
# Contador
i=1
while [ $i -le 5 ]; do
    echo "Vuelta $i"
    i=$(( i + 1 )) # sin esta línea: bucle infinito
done

# Validar una entrada hasta que sea correcta
read -p 'Nota (0-10): ' nota
while [ $nota -lt 0 ] || [ $nota -gt 10 ]; do
    read -p 'Inválida, repite: ' nota
done

# Bucle infinito controlado (lo usé con nohup)
while true; do
    date
    sleep 5
done
```

- **break**: sale del bucle inmediatamente.
- **continue**: salta a la siguiente vuelta sin ejecutar el resto del cuerpo.
- **until [cond]**: lo contrario de **while**; repite **hasta que** la condición sea verdadera.

2.2 while para leer un archivo línea a línea

```
while IFS=, read -r campo1 campo2 campo3
do
    echo "$campo1 - $campo2"
done < fichero.csv
```

Piezas que debo saber explicar una por una:

- **read -r**: lee **una línea** cada vez. La **-r** evita que se interpreten las barras \.
- **IFS=,**: cambia el separador de campos (por defecto es el espacio). Sin esto el CSV no se parte por comas.
- **Variables** (**campo1...**): reciben cada columna. Si sobran columnas, la última variable se queda con todo el resto.
- **done < fichero**: es lo que alimenta el bucle. Cuando se acaba el archivo, **read** devuelve error y el **while** termina solo.

Mi caso real: crear usuarios desde usuarios.csv

```
contador=0
while IFS=, read -r nombre apellido dni telefono correo usuario
do
    sudo useradd $usuario -m -s /bin/bash -g alumnos_sn 2>/dev/null
    if [ $? -eq 0 ]; then
        contador=$(( contador + 1 ))
        echo "Usuario $usuario CREADO correctamente"
    else
        echo "El usuario $usuario YA EXISTIA"
    fi
done < $1

echo "Se han creado $contador usuarios"
echo "$(( $lineas - $contador )) ya estaban creados"
```

Cada fila de **usuarios.csv** tiene **6 campos** (nombre, apellido, DNI, teléfono, correo, usuario), y por eso **read** lleva 6 variables. Por ejemplo:

Juan,Martínez,12345678A,611223344,juan.martinez@academia-sn.com,jmartinez78_sn.

Patrón contador: **contador=0** antes del bucle; **contador=\$((contador + 1))** dentro, cuando algo sale bien; el resumen **después** del bucle. Es el patrón que más se repite en mis scripts de usuarios.

Variantes

```
# Sin separador especial: línea completa
while read -r linea; do
    echo "$linea"
done < archivo.txt

# Con pipe (funciona, pero con trampa)
cat archivo | while read -r linea; do
    ...
done
```

Trampa del pipe: con **cat archivo | while ...** el bucle corre en una *subshell*. Las variables que modifiques dentro (como **contador**) **no existen fuera**. Por eso es mejor **done < archivo**.

2.3 Bucle for

```
for variable in lista
do
    comandos
done
```

a) Sobre una lista de elementos

```
for fruta in manzana pera uva; do
    echo "Me gusta la $fruta"
done
```

b) Sobre un rango numérico

```
for i in {1..3}; do                # mi animación de "SALIENDO..."
    echo -n '.'
    sleep 0.5
done

for i in {1..10..2}; do echo $i; done      # 1 3 5 7 9 (paso 2)
for i in $(seq 1 $n); do echo $i; done     # cuando el límite es una variable
for (( i=1; i<=5; i++ )); do echo $i; done # estilo C
```

`{1..$n}` **no funciona** con variables, porque la expansión de llaves se hace antes que la de variables. Usa `seq 1 $n` o el estilo C.

c) Sobre ficheros (comodines)

```
for f in *.deb; do
    echo "Instalando $f"
    sudo dpkg -i "$f"
done
```

d) Sobre el resultado de otro comando

```
for u in $(cut -d, -f6 usuarios.csv); do      # lista de usuarios del CSV
    id $u
done

for p in $(pidof yes); do kill $p; done
```

`for` separa por **espacios y saltos de línea**. Si cada línea contiene espacios (por ejemplo nombres compuestos), usa `while read` en su lugar.

2.4 Errores clásicos con bucles

- Olvidar modificar la variable de la condición → **bucle infinito**.
- Olvidar `do` o `done`.
- Quitar los espacios dentro de `[ ]`: `[ $a -eq 1 ]`.
- Usar `=` con números (usa `-eq`) o `-eq` con texto (usa `==`).
- Olvidar `done < fichero` en un `while read`: el script se queda esperando teclado.
- Cambiar variables dentro de un `while` alimentado por pipe y esperar que sobrevivan fuera.

2.5 Bucles dentro de mis scripts: mapa rápido

Script	Bucle	Para qué
script-usuarios-DRV.sh	<code>while IFS=, read -r ... done < \$1</code>	Crear cada usuario del CSV.
script-usuarios-DRV.sh	<code>for i in {1..3}</code>	Animación «SALIENDO...».
script-eliminar-usuarios-DRV.sh	<code>while [\$opcion -ne 0]</code>	Menú principal.
script-eliminar-usuarios-DRV.sh	<code>while IFS=, read -r ... done < \$1</code>	Comprobar y eliminar cada usuario.
script-investigacion.sh	<code>while [\$opcion -ne 0]</code>	Menú de actividad de usuarios.
gestionappslinux.sh	<code>while [\$opcion -ne 0] y for i in {1..3}</code>	Menú principal y animaciones.

3. Fallos de mis scripts (por si piden depurar)

Estos son los errores reales que encontré en mi código. Son justo el tipo de cosa que se pregunta en un examen: «¿qué falla aquí y cómo lo arreglas?».

1. Comillas simples no expanden variables — *script-usuarios-DRV.sh*

Mal:

```
echo '$usuario:$apellidos$DNI' | sudo chpasswd
```

Por qué: Las comillas simples **no expanden** variables, así que la contraseña sería literalmente el texto `$usuario:$apellidos$DNI`. Además la variable se llama `$apellido` y `$dni` (minúsculas).

Bien:

```
echo "$usuario:$apellidodni" | sudo chpasswd
```

2. `[ -z $? ]` nunca es verdadero — *script-investigacion.sh*

Mal:

```
who 2>/dev/null
if [ -z $? ]; then ...
```

Por qué: `$?` siempre contiene un número (0, 1...), nunca está vacío. No se está comprobando la salida de `who`.

Bien:

```
if [ -z "$(who)" ]; then
    echo "Nadie conectado"
fi
```

3. El último `elif` nunca se alcanza — *script4.sh*

Mal:

```
if [ $nota -ge 5 ]; then ...
elif [ $nota -lt 5 ]; then ...
elif [ $nota -lt 0 -o $nota -gt 10 ]; then ... # inalcanzable
```

Por qué: Toda nota es `o >= 5 o < 5`, así que la tercera condición nunca se evalúa. **La validación debe ir primero.**

Bien:

```
if [ $nota -lt 0 -o $nota -gt 10 ]; then echo "Nota inválida"
elif [ $nota -ge 5 ]; then echo "Aprobado"
else echo "Suspenso"; fi
```

4. Variable equivocada en la comparación — *script6.sh*

Mal:

```
elif [ $num2 -gt $num1 ] && [ $num1 -gt $num3 ] # compara num1 con num3
```

Por qué: El 2º número debe compararse con el 3º, no el 1º con el 3º.

Bien:

```
elif [ $num2 -gt $num1 ] && [ $num2 -gt $num3 ]
```

5. Año bisiesto incompleto — *script8.sh*

Mal:

```
if [ $((year % 4)) -eq 0 ]; then ...
```

Por qué: Un año es bisiesto si es divisible por 4 **y no** por 100, **o** si lo es por 400. Así 1900 no es bisiesto pero 2000 sí.

Bien:

```
if [ $((year%4)) -eq 0 ] && [ $((year%100)) -ne 0 ] || [ $((year%400)) -eq 0 ]; then ...
```

6. Variable sin asignar — *parametros2.sh*

Mal:

```
parametros=$*  
... $arch ...      # $arch nunca se definió
```

Por qué: `$arch` está vacía. Hay que asignarla desde el parámetro recibido.

Bien:

```
arch=$1
```

7. `grep` por subcadena — *script-eliminar-usuarios y script-investigacion*

Mal:

```
cat /etc/passwd | grep $usuario
```

Por qué: Busca la cadena en **cualquier parte** de la línea: `ana` coincidiría con `mariana`, e incluso con el directorio o el comentario.

Bien:

```
getent passwd "$usuario" >/dev/null  
# o bien: grep -w "^$usuario:" /etc/passwd
```

8. Variables sin comillas dentro de `[ ]` — *casi todos los scripts*

Mal:

```
if [ -f $archivo ]; then ...
```

Por qué: Si `$archivo` está vacía o tiene espacios, el test se rompe (`[ -f ]` o demasiados argumentos).

Bien:

```
if [ -f "$archivo" ]; then ...
```

9. Comando mal escrito — *script-eliminar-usuarios-DRV.sh*

Mal:

```
cho -e "${rojo}Debes ELEGIR una OPCIÓN del 0-3${normal}"
```

Por qué: Es `echo`, no `cho`. Daría «command not found».

Bien:

```
echo -e "${rojo}Debes ELEGIR una OPCIÓN del 0-3${normal}"
```

Otros detalles que también cuentan

- `informacion()` usa `$lineas`, que se define fuera de la función. Funciona porque en Bash las variables son globales por defecto, pero sería más limpio pasarla como parámetro.
- En el menú de *script-DRV.sh* la segunda pregunta dice «¿A qué propietario...?» dos veces, cuando la segunda debería ser «¿A qué grupo...?».

- En `gestionappslinux.sh` mis funciones se llaman `apt` y `dpkg`, igual que los programas. Una función con el mismo nombre **tapa** al comando: funciona porque dentro uso `sudo apt ...` (sudo ejecuta el programa real), pero un `apt update` sin `sudo` dentro de la función se llamaría a sí misma en bucle. Es más seguro llamarlas `menu_apt` y `menu_dpkg`.
- `apt autoremove $app` no necesita el nombre del paquete.
- Al final de `permisos.sh` hay `cd + cd Escritorio/` antes del `chgrp`, lo cual cambia de carpeta y rompe las rutas relativas.

4. Chuleta final y plan de repaso

4.1 Plantillas que debo saber escribir de memoria

Validar parámetros, archivo y root

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Uso: $0 fichero.csv"; exit 1
fi
if [ ! -f "$1" ]; then
    echo "El fichero no existe"; exit 1
fi
if [ $UID -ne 0 ]; then
    echo "Ejecuta con: sudo $0 $1"; exit 1
fi
```

Menú con while + case + funciones

```
function menu(){
    clear
    echo "0) Salir"
    echo "1) Opción 1"
}
opcion=1
while [ $opcion -ne 0 ]; do
    menu
    read -p "Elige: " opcion
    case $opcion in
        0) echo "Adiós" ;;
        1) echo "Hola" ; read -p "ENTER..." ;;
        *) echo "Opción incorrecta" ; read -p "ENTER..." ;;
    esac
done
```

Leer CSV, crear usuarios y contar

```
contador=0
while IFS=, read -r nombre apellido dni telefono correo usuario; do
    if sudo useradd "$usuario" -m -s /bin/bash 2>/dev/null; then
        echo "$usuario:${apellido}${dni}" | sudo chpasswd
        contador=$(( contador + 1 ))
    else
        echo "$usuario ya existía"
    fi
done < "$1"
echo "Creados: $contador"
```

Confirmación S/N

```
read -p "¿Seguro? [S/N]: " si_no
if [ "$si_no" == "S" ] || [ "$si_no" == "s" ]; then
    echo "Procediendo..."
elif [ "$si_no" == "N" ] || [ "$si_no" == "n" ]; then
    echo "Cancelado"
else
    echo "Debes elegir S o N"
fi
```

Comprobar el resultado de un comando

```
mkdir "$dir" 2>/dev/null
if [ $? -eq 0 ]; then echo "Creado"; else echo "Error"; fi

# Forma abreviada: el if evalúa directamente el comando
if mkdir "$dir" 2>/dev/null; then echo "Creado"; fi
```

4.2 Tabla resumen de parámetros especiales

Variable	Contenido
\$0	Nombre del script.
\$1 \$2 ...	Parámetros posicionales.
\$#	Número de parámetros.
\$* / @\$	Todos los parámetros.
\$?	Código de salida del último comando (0 = OK).
\$\$	PID del propio shell o script.
\$!	PID del último proceso lanzado en segundo plano.
\$UID	UID del usuario (0 = root).

4.3 Plan de repaso para esta noche (por prioridad)

#	Qué repasar	Por qué
1	<code>while IFS=, read -r ... done < \$1</code> y el patrón contador.	Es mi script más complejo y el más típico de examen.
2	Validaciones: <code>##</code> , <code>-f</code> , <code>\$UID</code> , y por qué <code>\$?</code> va justo después del comando.	Aparecen en todos los scripts de UD5.
3	<code>useradd</code> con todas sus opciones (<code>-c -d -m -s -g -G -e -f</code>) y <code>userdel -r</code> .	Pregunta casi segura.
4	<code>case</code> dentro de un <code>while</code> de menú, con funciones.	Estructura base de mis tres scripts grandes.
5	<code>usermod</code> , <code>groupmod</code> , <code>groupdel</code> , <code>passwd -S</code> , <code>chage</code> .	Salen en los ejercicios de usuarios I y II.
6	Los 9 fallos del capítulo 3.	Por si piden depurar código.
7	<code>ps</code> , <code>kill</code> (señales), <code>nice</code> , <code>jobs/fg/bg</code> , <code>nohup</code> , <code>timers</code> .	Ejercicios de UD6.

Consejo para el examen: si un script falla, comprueba por este orden: (1) ¿espacios dentro de `[ ]`? (2) ¿comillas simples vs. dobles? (3) ¿`do/done`, `then/fi`, `;;/esac` bien cerrados? (4) ¿variable bien escrita y asignada? (5) ¿`$?` leído justo tras el comando?